

Beeline Tariff Marketing Campaigns Case

ТЗ для участников хакатона

23 441 абонент. 100 000 у.е. бюджета. 20 пилотов. 9 часов. Постройте агента, который решит, кому предложить сменить тариф — и не разорит компанию по дороге.

⚠ **Все данные и цифры в этом кейсе синтезированы специально для хакатона.** Это не выгрузка из систем Beeline и не реальные показатели компании: объёмы, тарифы, суммы, доли и эффекты сгенерированы для учебной задачи. Любые выводы о реальном бизнесе оператора на их основе некорректны.

1. Название

Агент управления тарифными маркетинговыми кампаниями.

2. Проблема

Beeline зарабатывает на тарифах, и ключевая метрика здесь — **ARPU** (средняя выручка на абонента). Чтобы её растить, компания постоянно проводит маркетинговые кампании: выбирает сегмент абонентов, предлагает им перейти на другой тариф и доносит это через какой-то канал связи.

Проблема в том, что кампания «вслепую» уходит в минус. Значительная часть смен тарифа заканчивается **downsell** — абонент после перехода платит *меньше*, чем платил до; ещё часть переходов не меняет ничего. То есть заметная доля контактов — это деньги без результата или прямой ущерб выручке. Сам контакт тоже не бесплатен: звонка на весь бюджет хватит лишь на 625 человек из 23 тысяч.

Сегодня процесс уже data-driven: сегментация по данным потребления, ML-модели склонности, проверка гипотез на истории. Но решение «какие кампании запустить и как поделить бюджет» принимает аналитик вручную — это долго, и часть вариантов просто не успевают рассмотреть.

3. Пользователь

Основной пользователь: аналитик маркетинга, планирующий кампании на следующий месяц.

Ключевой сценарий: запускает агента на текущей базе → агент сам исследует данные, проверяет гипотезы пилотными кампаниями и возвращает готовый план: какие кампании, на какие сегменты, через какие каналы и как потратить бюджет.

4. Задача

Разработать **агентскую систему**, которая выбирает маркетинговые кампании и выдаёт план из **максимум 10 кампаний**. Одна кампания — три решения:

	Решение	Из чего выбирать
Кого	сегмент абонентов	ARPU, потребление данных и звонков, текущий тариф
Что	целевой тариф	любой из 21
Как	канал коммуникации	push / SMS / реклама / звонок

Главная сложность — неполная информация. Выданная история описывает *другую* выборку абонентов, а не ту, на которой считается результат. Как поведёт себя целевая аудитория, заранее неизвестно. Узнать можно только **пилотом** — запустить кампанию на малой выборке и посмотреть.

Но пилот врёт: **пилот на 30 клиентах назовёт убыточным умеренно прибыльный сегмент примерно в каждом четвёртом случае**. На 200 клиентах — в одном из двадцати пяти, но и стоит он в семь раз дороже, из того же бюджета, что и сами кампании. При этом стратегия без разведки приносит **в ~15 раз меньше**, чем знающая истинные эффекты: разведка — основной источник результата.

5. Вход → выход

Вход — объект среды `env`, который получает метод `act()`:

```
class Agent:
    def act(self, env) -> list[dict]:
        # env.customer_profile — DataFrame аудитории (23 441 абонент)
        # env.tariffs, env.channels
        # env.remaining_budget, env.remaining_contacts, env.pilots_left
        # env.run_pilot(target_tariff=..., channel=..., n_customers=...,
        #               filter_arpu_segment=..., filter_current_tariff=...)
        # env.pilot_history — ваши пилоты и их результаты
        return [...] # до 10 кампаний
```

Выход — список словарей. Обязательны `target_tariff` и `channel`, фильтры опциональны (пропущен = не фильтровать):

```
{"campaign_name": "Premium Upsell", "filter_arpu_segment": "HIGH",
 "filter_data_segment": "HEAVY", "filter_current_tariff": "tariff_4;tariff_8",
 "target_tariff": "tariff_10", "channel": "sms"}
```

Как считается результат. Эффект кампании — это **процент** от ARPU конкретного абонента, а не фиксированная сумма: один и тот же переход на дорогом абоненте приносит больше.

```
Чистый результат = Прирост ARPU (по уникальным абонентам)
                  - Затраты на коммуникацию (контакты × стоимость канала)
```

Если не делать ничего, аудитория принесёт **150 641 084** (сумма колонки `predicted_arpu`). Побеждает тот, чей чистый результат больше. В зачёт идут **и пилоты, и финальные кампании** — пилотные контакты тоже реальные.

Канал	Стоимость контакта	Эффективность
push	бесплатно	низкая (×0.50)
sms	4 у.е.	средняя (×0.65)
digital_ads	22 у.е.	выше среднего (×0.85)
call	160 у.е.	высокая (×1.20)

6. Данные

Всё в пакете участника. Данные синтетические (см. оговорку в начале): ~23 тысячи абонентов, период 2026 год, тарифы обозначены `tariff_1...tariff_21` и не отражают действующую линейку, суммы — в условных единицах.

Файл	Что внутри
customer_profile.csv	аудитория кампаний: 23 441 абонент, текущий тариф, сегменты, поведение, <code>predicted_arpu</code>
data/change_tariff.csv	история 14 824 смен тарифов с ARPU до и после
data/traffic.csv	потребление (минуты, SMS, трафик, устройство) по месяцам
data/arpu_monthly.csv	месячная выручка по абонентам
data/dict_tariff.csv	параметры 21 тарифа (цена, пакеты)
tariff_dictionary.csv, feature_dictionary.csv	описания тарифов и всех колонок

Сегменты в `customer_profile.csv`:

Колонка	Значения	Порог
arpu_segment	LOW / MID / HIGH	по <code>ARPU_3m_avg</code> : <1000, 1000–5000, >5000
data_segment	NON_USER / LITE / HEAVY	0 МБ/мес, 0–2000, >2000
call_segment	LOW / MEDIUM / HIGH	<100 мин/мес, 100–400, >400

Готовой таблицы «тариф A → тариф B = +X ARPU» в комплекте нет — иначе задача свелась бы к сортировке одной колонки. Оценку эффекта агент строит сам.

7. Must have

Без этого решение не засчитывается. Всё проверяется одной командой `python local_eval.py`:

1. **Файл `agent.py` с классом `Agent` и методом `act(env)`**, запускается без ошибок. *Проверка:* `local_eval.py` отработывает и печатает результат.
2. **От 1 до 10 корректных кампаний** с существующими тарифами и каналами. *Проверка:* в выводе нет строк «Кампания ... отброшена».
3. **Агент использует пилоты** (`env.run_pilot`) и опирается на их результаты, а не действует по зашитым константам. *Проверка:* в выводе `Пилотов проведено > 0`, логика видна в коде.
4. **Агент укладывается в лимиты** бюджета, охвата и числа пилотов. *Проверка:* прогон завершается без превышений.
5. **Файл `submission.csv`** из команды `python make_submission.py`. *Проверка:* файл есть и воспроизводится нашим запуском.

8. Опционально

Дают преимущество, но не блокируют сдачу:

- **Учёт неопределённости:** агент оценивает не только среднее по пилоту, но и его надёжность (пилот на 150 клиентах точнее, чем на 30).
- **Адаптивная разведка:** размер и число пилотов зависят от уже выясненного, а не заданы жёстко.
- **Осмысленный выбор канала** под ценность сегмента, а не один канал на всё.
- **Устойчивость:** результат не разваливается при неудачной серии пилотов (`python local_eval.py --runs 10` — знак не должен скакать).
- **LLM в контуре принятия решений** — разрешено и приветствуется.

9. Ограничения

Лимиты (действуют одновременно):

- максимум **10 кампаний**;
- максимум **5 000 абонентов на кампанию** (сверх — не засчитывается);
- максимум **15 000 контактов** суммарно, включая пилотные;
- бюджет **100 000 у.е.** на все контакты, включая пилотные;
- максимум **20 пилотов**, по 10–200 абонентов в каждом;
- **каждый абонент засчитывается один раз** — по лучшей для него кампании. Продублировать удачную кампанию и умножить эффект нельзя, а лишние контакты всё равно стоят денег.

Нельзя:

- лезть во внутренности среды в обход пилотов (`__closure__`, `gc`, чтение файлов организатора и подобное) — **результат аннулируется**, код проверяется автоматически;
- хардкодить секреты: ключ к LLM берите из `os.environ["OPENAI_API_KEY"]`, на судействе его подставим мы.

Обязательно учесть:

- агент работает **не дольше 10 минут**, включая все вызовы модели;
- LLM API может ответить ошибкой — оборачивайте вызовы в `try/except` и имейте запасную логику. Если агент упадёт, проведённые пилоты всё равно пойдут в зачёт, но результат будет отрицательным;
- эффекты на судействе **другие**, чем в мок-среде: настраивайте логику, а не константы под мок.

10. Артефакты

Артефакт	Обязателен
<code>agent.py</code>	да
<code>submission.csv</code> (генерируется <code>make_submission.py</code>)	да
<code>requirements.txt</code>	если нужны библиотеки
README с описанием подхода	нет, но помогает при разборе

Проверка себя перед сдачей: `python local_eval.py` (прогон + отчёт), `--runs 10` (устойчивость по разным seed), `python make_submission.py`.

`local_eval.py` считает результат **тем же кодом**, что и на судействе, но на мок-эффектах — показывает механику и поведение агента, а не будущий балл. Пример для старта — `agent_template.py` (намеренно слабый, обыграть несложно).